

Prompt engineering template

Copy this template and fill in the bracketed sections. Remove any blocks you don't need — not every prompt requires all seven.

Full template

[ROLE]

You are a [role/title] at [company type or context]. You [key trait that shapes tone or approach].

[CONTEXT]

Context: [Background the model needs that it wouldn't otherwise have – product details, audience info, situation, constraints of the real world, etc.]

[EXAMPLES]

Here are examples of the style/format I want:

"[Example output 1]"

"[Example output 2]"

[FORMAT]

Structure your response as:

- [Section 1 – name and any length/style constraint]
- [Section 2]
- [Section 3, optional]

[TASK]

[Your clear, specific ask – one or two sentences.]

[CONSTRAINTS]

Do not: [thing to avoid 1], [thing to avoid 2].

Do not use words like "[banned word 1]", "[banned word 2]".

Keep the total response under [X] words.

[REASONING]

Before writing, think through: [the key question the model should answer in its head first – e.g. "what is the user's biggest pain point this solves?"]

Minimal version (3-block starter)

Good for quick tasks where you just need role + task + constraints.

```
[ROLE]
You are a [role] with expertise in [area]. Your tone is [adjective].

[TASK]
[Your specific ask.]

[CONSTRAINTS]
Do not: [key things to avoid].
Keep the response under [X] words / in [format].
```

Component reference

Component	What it does	When to include
Role	Sets tone, vocabulary, and perspective	Almost always
Context	Gives facts the model can't guess	Whenever the task is domain-specific
Examples	Calibrates style and format better than words can	When format or voice really matters
Format	Removes structural ambiguity	Whenever the output has a defined shape
Task	The actual ask — keep it short	Always
Constraints	Prevents the most predictable failure modes	When you know what you don't want
Reasoning	Forces the model to think before it writes	For complex, multi-step, or analytical tasks

Tips

- **Specificity beats length.** A sharp 3-block prompt beats a vague 10-block one.
- **Iterate like debugging.** Wrong tone? Fix the role. Wrong structure? Fix the format. Wrong logic? Add reasoning.
- **Negative constraints work.** Banning specific words ("don't use 'seamless'") forces more original output.
- **Examples > descriptions.** If you can show one good output, do that instead of explaining what you want.
- **Keep the task block short.** Everything else does the narrowing — the task just fires the trigger.

Worked example (changelog entry)

```
[ROLE]
You are a senior product writer at a B2B SaaS company. You write clearly and warmly,
```

always centering the customer's benefit over technical detail.

[CONTEXT]

Context: We just shipped bulk CSV export for our analytics dashboard. Users can now select any date range and export up to 500,000 rows of raw event data. Previously, exports were capped at 10,000 rows with no date filtering. Our users are data analysts and ops teams who use this data for offline reporting in Excel or BI tools.

[EXAMPLES]

Here are two past changelog entries in our style:

"Faster dashboard loads – Reports now load up to 3x faster thanks to query optimisation on our end. No action needed on yours."

"Custom date ranges on all charts – You can now set any date window on every chart in the dashboard, not just the presets."

[FORMAT]

Structure the entry as:

- Headline (bold, max 8 words, benefit-first)
- Body (2–3 sentences max)
- One optional "how to find it" sentence

[TASK]

Write a changelog entry for the bulk CSV export feature.

[CONSTRAINTS]

Do not use: "performant", "robust", "seamless", or "powerful".
Do not mention internal technical implementation.
No exclamation marks.
Keep the total entry under 80 words.

[REASONING]

Before writing, think through: what is the user's biggest pain point this solves,
and what is the single most important thing they need to know?